

Relazione Tecnica: Modello AI per la Zero Trust Architecture

Analisi del Temporal Graph Network (Graphagate)

Capitolo 1

Il modello Graphagate: Temporal Graph Network

1.1 Introduzione

Nel contesto di una Zero Trust Architecture (ZTA), il processo di autorizzazione continua richiede meccanismi dinamici per rilevare deviazioni dal comportamento abituale e segnali di compromissione. Il modello integrato con l'Orchestrator OPA è basato su una *Temporal Graph Network* (TGN), progettata specificamente per il *self-supervised anomaly detection* su stream di traffico continuo.

Ogni accesso (una richiesta IP/device verso una risorsa) rappresenta un arco temporale all'interno di un grafo in evoluzione. Il modello mantiene uno stato ricorrente per ciascuna entità (utente o risorsa) e analizza i vicini temporali recenti per assegnare uno score di anomalia in tempo reale, consentendo di bloccare le richieste ostili ed esfiltranti.

1.2 Architettura del sistema

L'architettura del modello combina la memoria ricorrente tipica dei TGN (Rossi et al., 2020) con un gestore del vicinato ottimizzato (bounded in RAM) e uno *scorer* a due teste che permette di analizzare le anomalie su due fronti: contestuale/policy e strutturale.

1.2.1 Componenti principali

- *TGNMemory*: struttura basata su GRU che aggiorna lo stato storico di ciascun nodo con i messaggi degli eventi in ingresso. La dimensione è impostata a 256 per gestire le impronte comportamentali.
- *Hashed Identity*: identità dei nodi mappate tramite funzioni di hash (`hash(URI) % hash_buckets`) per garantire totale *induttività* e scalabilità in presenza di entità mai viste prima.
- *MessageNeighborLoader*: buffer in RAM che memorizza gli ultimi vicini temporali di ciascun nodo (es. `neighbor_size=30`). Lavora con lookup in tempo $O(1)$ su matrici preallocate ed elimina la necessità di lenti database a grafo.
- *GraphAttentionEmbedding*: rete multi-hop con *TransformerConv* per calcolare l'embedding attendendo sui vicini storici, concatenando l'identità alla storia temporale.
- *Static Node Features*: vettore a 16 dimensioni associato a ciascun nodo, che codifica attributi ZTA essenziali come *device tier*, *clearance* e *trust score*, oltre a metriche estese introdotte nello schema v3 quali il *resource risk* (per innalzare la sensibilità su asset

critici) e il *source internal/external flag* (per distinguere reti RFC1918 da IP pubblici; ablatabile e *disattivato di default* nella configurazione corrente, poiché il roaming esterno benigno può mascherare il furto di credenziali — la coppia *external*→*device* che esporrebbe l’attaccante).

- *History Features* (esplicite, non circolari): per ogni arco valutato si calcolano tre contatori causali e *benign-gated* (aggiornati solo su traffico approvato, anti-poisoning): $\log(1+\text{pair_count})$, $\log(1+\text{src_count})$, $\log(1+\text{dst_count})$. A questi si aggiunge una seconda tripletta identica per le coppie *aux_src*→*dst* (es. per validare la *device habituality*), portando la dimensione a 6. Misurano quanto è abituale quell’accesso: una coppia mai vista da una sorgente attiva è la *novelty cue* del movimento laterale, che, combinata con la memoria temporale, porta il maggior contributo (ablation multi-seed a 3 seed: +0.163 AUC, il segnale dominante del laterale).
- *Dual Head Scorer*: lo score di un arco è la *somma* di due logit complementari, $\text{logit} = \text{feat_logit} + \text{struct_logit}$.
 - *Feature head* (*LinkPredictor*, MLP a 3 strati): consuma gli embedding dei due endpoint, il messaggio dell’evento, gli attributi statici con hashed-identity, le codifiche di recency (Δt coppia, Δt sorgente) e le *history features*. Cattura anomalie *contestuali* e di *policy* e veicola la *novelty cue* del laterale.
 - *Structural head*: similarità coseno (scalata da una temperatura apprendibile) delle proiezioni non-lineari degli embedding. *Nota*: l’ablation a 3 seed la mostra *marginale* sul laterale (0.882 con vs 0.875 senza, +0.007 AUC); su questo stream il segnale è già portato dalle *history features* e dalla memoria temporale. La testa è comunque *mantenuta* come segnale complementare a basso costo: l’ottimizzatore le assegna una temperatura non banale ($\text{struct_scale} \approx 4.7$ nel checkpoint addestrato), dunque è *attiva ma ridondante*, non inutilizzata. Poiché nel task sintetico il movimento laterale coincide quasi per costruzione con la *pair-novelty* - catturata direttamente dalle *history features* - l’ablation ne sottostima probabilmente il contributo: il suo valore su topologie reali (dove il laterale ha struttura di grafo oltre alla semplice novità della coppia) e in regime di *cold-start* o di *poisoning* dei contatori, dove il coseno sugli embedding è un segnale indipendente dai contatori espliciti, è plausibile ma *non ancora verificato*.
- *Kill-chain Precursor* (prior di serving, non addestrato): il movimento laterale segue tipicamente un alert di recon (Snort / anomalia rilevata) sulla *stessa* entità, ma il gate predict-then-update scarta quel precursore dalla memoria TGN. Si mantiene quindi uno stato per-entità **recent_alert** e si applica un fattore *moltiplicativo* allo score, $1 + \text{max_boost} \cdot 0.5^{\Delta t / \text{half_life}}$ (default $\text{max_boost}=2$, $\text{half_life}=600\text{s}$). Moltiplicativo per costruzione: alza uno score già sospetto sopra soglia senza trasformare in falsi positivi gli score benigni ≈ 0 . Il suo contributo è però *marginale* (ablation multi-seed a 3 seed: +0.013 AUC), comparabile a quello della testa strutturale. Non costituisce un input addestrato (il training solo-benigno lo renderebbe una feature morta).

1.3 Fase di addestramento e calibrazione

Il modello viene addestrato con un approccio *self-supervised* unicamente su traffico benigno, senza mai essere esposto a etichette di anomalia (unsupervised anomaly detection). Lo stream temporale sintetico generato viene suddiviso cronologicamente in tre subset: addestramento (70%), validazione (10%) e test (20%).

L’obiettivo di apprendimento per la rete è il *link prediction*: il TGN è istruito per massimizzare lo score assegnato ad eventi realmente accaduti nella baseline benigna e minimizzare

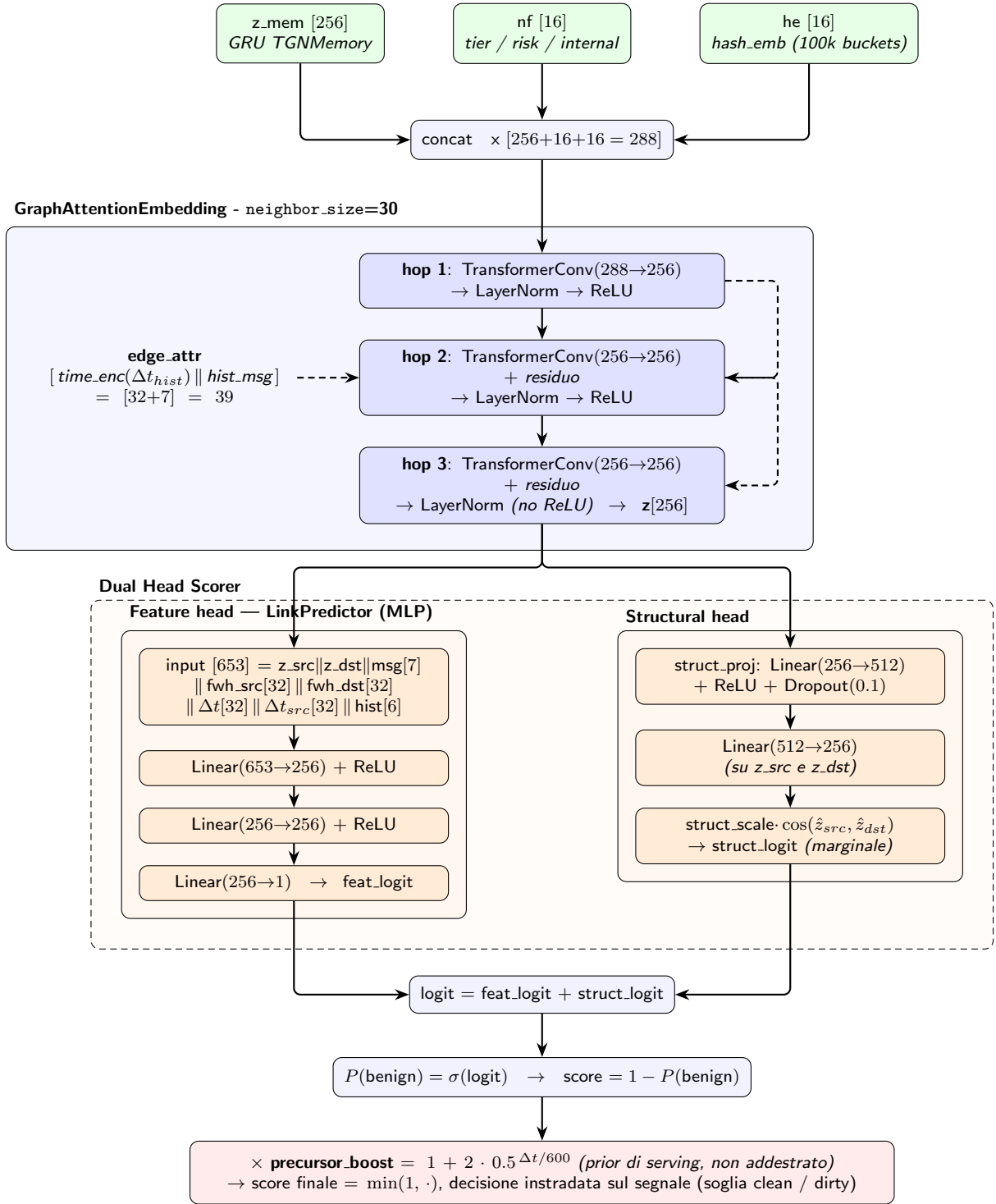


Figura 1.1: Schema dei layer dove i moduli interni sono esplicitati per i due di alto livello (GAE ed il Dual Head Scorer): i tre hop dentro *GraphAttentionEmbedding* e le due teste (*Feature/Structural*) dentro il *Dual Head Scorer*. Tutti i *TransformerConv* usano **heads=4**, **dropout=0.1** e concatenano **edge_attr** (alimenta ogni hop). Lo score grezzo $1 - P(\text{benign})$ è moltiplicato dal prior di kill-chain (con cap a 1) e confrontato con la soglia instradata sul segnale; a serving lo score dell'evento è il massimo sui suoi archi (accesso e binding).

contemporaneamente lo score associato ad eventi artificialmente manipolati e generati a runtime (*negative sampling*).

1.3.1 Negative sampling e funzione di loss

Il processo di ottimizzazione (guidato dall'algoritmo AdamW) somma tre termini self-supervised. La componente dominante è una loss di ranking InfoNCE, allineata all'Average Precision, affiancata da due ancoraggi BCE:

- *Structural Negatives* (InfoNCE, $K=5$ random target): per ogni positivo benigno la sorgente è accoppiata a K destinazioni *uniformemente casuali* sull'intero id-range delle destinazioni. L'obiettivo InfoNCE impone che, tra $\{\text{dst-vera}, K \text{ dst-casuali}\}$, la dst-vera ottenga il logit più benigno *dato lo storico della sorgente*. La rete apprende così $P(\text{dst} | \text{src}, \text{storia})$: un accesso a bassa verosimiglianza (movimento laterale o violazione di policy) riceve uno score elevato.
- *Positive BCE Anchor*: i logit dei positivi benigni vengono spinti verso "benigno". L'InfoNCE fissa solo l'ordine relativo; questo ancoraggio mantiene gli score su una scala assoluta, così che la soglia calibrata sull'FPR resti significativa.
- *Contextual Negatives* (Gaussian Feature Noise): mantenendo intatta la destinazione, al messaggio dell'evento si somma rumore gaussiano ($\sigma=0.5$) e si addestra la *Feature Head* a marcarlo come anomalo. Si tratta di un meccanismo distinto rispetto alle anomalie contestuali della valutazione (randomizzazione discreta dei segnali 0/1), così il modello non riceve la corruzione del test.

1.3.2 Calibrazione della soglia (cost-sensitive instradata sul segnale)

Il TGN non emette una classificazione netta ma uno score continuo confrontato con una soglia, calibrata via *replay* sul 10% di validazione held-out (lo split di test non è mai toccato in calibrazione). Poiché a serving la classe vera è ignota ma il *segnale di edge* (JA3/Snort/sensori) è osservabile, si calibrano e si instradano due soglie:

- *Soglia signal-dirty* (eventi il cui segnale già scatta): il classico quantile al Target FPR dell'1% sugli score benigni. Questi eventi sono già intercettati dalla cheap rule baseline, quindi si mantiene la soglia conservativa.
- *Soglia signal-clean* (dove il laterale è indistinguibile dal benigno se non per il pattern temporale): soglia *cost-sensitive* che minimizza $\text{cost_ratio} \cdot \text{FN} + \text{FP}$ (default `cost_ratio=20`: un laterale mancato costa molto più di un falso allarme ri-sfidabile dall'orchestrator). Un *cap* esplicito sull'FPR del flusso clean (`clean_fpr_cap=0.05`) impedisce che la spinta sul recall faccia esplodere i falsi positivi — necessario perché anche con lateral AUC ~ 0.90 (singola run) il trade-off recall/FPR sul flusso clean resta ripido (a recall laterale $\sim 79\%$ corrisponde un FPR benigno $\sim 13\%$).

A inferenza la decisione è instradata per-evento: `signal_dirty` $\Rightarrow$ soglia dirty, altrimenti soglia clean (cost-sensitive). Entrambe le soglie sono persistite ed esposte da `serve_api` (anche su `/health`). OPA resta il decisore finale allow/deny.

1.4 Tipologie di anomalie rilevate in produzione

In fase di valutazione o in ambiente live, l'architettura rileva 3 macro-categorie di attacco, dimostrando vantaggi misurabili rispetto a baseline più semplici (rule-based, Isolation Forest, One-Class SVM e GNN non temporale; cfr. Sezione 1.5):

1. *Policy Anomaly*: l’entità agisce al di fuori del proprio ruolo, clearance o device tier. Intercettata agevolmente dalla Feature head grazie alle feature statiche ZTA.
2. *Contextual Anomaly*: la connessione ha caratteristiche insolite o ha innescato sensori IPS. Rilevata dalla Feature head decostruendo le *edge features*.
3. *Lateral Movement*: l’accesso è formalmente autorizzato a livello di regole OPA, ma *non abituale*. Avviene tipicamente nel “pivoting”. Essendo indistinguibile sintatticamente dal traffico benigno, la classificazione ricade quasi del tutto sull’architettura implementata.

1.5 Valutazione sperimentale e confronto con le baseline

La valutazione è condotta sullo split di test (20% finale dello stream sintetico, `seed=42`) tramite replay per-evento attraverso l’esatto codice di serving (`num_events=200 000`, di cui il 20% finale = 40 000 eventi di test). Per quantificare il contributo *effettivo* della componente temporale, il TGN è confrontato con tre baseline non supervisionate addestrate sullo *stesso* traffico benigno, con identico split cronologico, identici segnali tabellari (contatori di storia + prior precursore), soglia calibrata all’1% di FPR e protocollo di valutazione (codice in `tests/baselines/`); una quarta baseline *supervisionata* (XGBoost) è riportata solo come upper-bound di riferimento:

- *Isolation Forest*: detector non relazionale sulle sole feature statiche per-evento (nessuna struttura di grafo) — rappresenta il “pavimento” privo di informazione strutturale (lateral AUC 0.639).
- *One-Class SVM*: controparte kernel non relazionale dell’Isolation Forest (sole feature per-evento). Pur con AUC aggregata competitiva (0.845), sul *lateral* resta a 0.632, confermando che il segnale temporale non è catturabile dalle sole feature statiche.
- *GNN non temporale*: ablation del TGN su grafo statico aggregato, con lo *stesso* curriculum di negative sampling de-circularizzato (InfoNCE su destinazioni casuali + rumore contestuale) e gli *stessi* contatori tabellari di storia, ma priva di memoria ricorrente e vicinato temporale. Isola la sola componente temporale (lateral AUC 0.468, $\approx$ caso).
- *XGBoost (supervisionato, fuori categoria)*: addestrato con le etichette, raggiunge lateral AUC 0.942. Non è un confronto equo col paradigma non supervisionato del TGN; è incluso solo per delimitare l’upper-bound raggiungibile avendo accesso alle label (assenti per costruzione nel nostro setting).

I risultati del confronto sono riportati nella Tabella 1.1.

1.5.1 Cost-sensitive routing

Per convertire la forte capacità di ranking (lateral AUC 0.900 su singola run; cfr. il caveat multi-seed in Tab. 1.1) in recall operativo, è stato implementato un routing cost-sensitive. Il flusso di eventi viene separato a runtime in *signal-dirty* e *signal-clean*. Sul flusso pulito, la soglia viene rilassata per minimizzare i falsi negativi dei movimenti laterali (configurabile tramite un parametro di `cost_ratio`). Sul test sintetico, ciò porta il recall laterale dal $\sim 13.0\%$ (soglia globale all’1% FPR, FPR benigno $\sim 2.3\%$) al $\sim 28.6\%$ (decisione instradata, FPR benigno $\sim 6.0\%$), con recall aggregato $\sim 70.3\%$.

Metrica (test set)	TGN	GNN non temp.	One-Class SVM	Isol. Forest
AUC aggregata	0.947	0.574	0.845	0.775
AP aggregata	0.870	0.569	0.793	0.628
Lateral — AUC	0.900	0.468	0.632	0.639
Lateral — Recall @1%FPR	0.130	0.139	0.047	0.010

Tabella 1.1: Confronto sul test set sintetico (replay per-evento de-circularizzato, `num.events=200000`, `seed=42`, soglia globale all'1% FPR; valori da *singola* run). Il vantaggio del TGN sul *lateral movement* emerge isolando il segnale temporale: la GNN statica, priva di memoria ricorrente ma provvista degli stessi contatori tabellari, resta a livello di caso (AUC 0.468), contro l'AUC 0.900 del TGN. Il recall laterale a soglia globale resta basso (13.0%) e l'apparente 13.9% della GNN è *spurio* (AUC≈caso con soglia permissiva, non un segnale reale); la sua conversione in recall operativo è affidata alla calibrazione cost-sensitive instradata (Sez. 4.3). *Nota metodologica*: il lateral AUC headline è da singola run (training GPU non-deterministico, ± 0.01 – 0.03 per run); la conferma *multi-seed* (3 seed, media $\pm$ dev.std) dà lateral AUC 0.882 ± 0.012 , coerente con l'headline a singola run. XGBoost *supervisionato* (vede le etichette, fuori categoria) raggiunge AUC 0.977 / lateral 0.942: è un upper-bound di riferimento, non una baseline non supervisionata comparabile.

1.6 Deployment e dettagli hardware

Le primitive di model-serving (estrazione memoria ed espansione grafo storico) operano asintoticamente a tempo costante $O(1)$. Il TGN mantiene tutto il suo stato dinamico e ricorrente nativamente in RAM. Ciò comporta due scelte architetturali di deployment: è inefficace una scalabilità orizzontale a monte (poiché più repliche frammenterebbero il database comportamentale delle entità) ed è fondamentale il dimensionamento asincrono.

Il sistema finale eroga un'infrastruttura single-worker fondata su **FastAPI** (gestione I/O asincrono per polling concorrentiale) che elabora gli eventi con una latenza end-to-end (round-trip HTTP client→servizio sulla rete container, percorso completo a 3 archi *source*→*device*→*user*→*dst*) di $P50 \approx 6.0$ ms ($P99 \approx 6.7$ ms); il percorso ablato senza *device-edge* scende a $P50 \approx 4.4$ ms ($P99 \approx 5.0$ ms). Misure su GPU RTX 5070 Ti (VRAM occupata ~ 2.4 GB), tramite test client di streaming live in regime *cold-start* (entità mai viste prima, prefisso `prod_`): un sanity-check d'induttività più che un punto operativo, poiché in cold-start il modello opera ad alto recall ma bassa specificità (lateral recall $\sim 47\%$ con *device-edge* vs $\sim 45\%$ senza, a fronte di una specificità benigna $\sim 76\%$ — FPR gonfiato dall'assenza di storico). Il binding di dispositivo dà un contributo marginale ma coerente; il punto operativo affidabile resta quello calibrato offline (Sez. 1.5).

Al fine di calcolare simultaneamente la backward-propagation e smaltire la mole della *Graph Attention*, il container isolato si appoggia alle seguenti specifiche hardware/software:

- container engine nativo agganciato a librerie NVIDIA CUDA 13;
- hardware accelerato su GPU con architettura NVIDIA Blackwell (testato su RTX 5070 Ti). La GPU accelera la *Graph Attention* multi-hop in inferenza e la backward-propagation in addestramento; lo stato ricorrente della `TGNMemory` e i ring-buffer del `MessageNeighborLoader` sono mantenuti con accesso $O(1)$ su tensori preallocati.

Nel momento di chiusura del framework (o tramite specifici endpoint API persistenti), la memoria serializzata di PyTorch (`tnn_checkpoint.pt` e `tnn_stats.json`) va copiata su sistema persistente per evitare cold-start traumatici.

1.7 Limiti e sfide implementative

1.7.1 La sfida del lateral movement

Senza l’ausilio delle etichette (unsupervised), il lateral movement non mostra tracce o indicatori palesi: è feature-identico a un accesso benigno non abituale. Le leve che lo rendono *ordinabile* (lateral AUC 0.90 su singola run — 0.882 ± 0.012 in media multi-seed a 3 seed su configurazione ridotta, coerente con la singola run; da ~ 0.47 della GNN statica) sono il vicinato temporale ampio (num.hops=3, neighbor.size=30, memory_dim=256) e soprattutto le *history features* esplicite (+0.163 AUC, il contributo dominante), affiancate dall’hashed-identity (+0.046 AUC); il precursore di kill-chain (+0.013 AUC) e la testa strutturale (+0.007 AUC) risultano invece *marginali* (ablation multi-seed). La conversione del ranking in recall operativo avviene poi con la soglia cost-sensitive instradata (Sezione 4.3): sul test sintetico il recall laterale passa da $\sim 13.0\%$ (soglia globale all’1% FPR, FPR benigno $\sim 2.3\%$) a $\sim 28.6\%$ (decisione instradata, FPR benigno $\sim 6.0\%$) — operating point regolabile via `cost_ratio/clean_fpr_cap`. Resta la classe più ostica (AP ~ 0.44), ma nettamente sopra le baseline (cfr. Sezione 1.5).

1.7.2 Anti-poisoning gate in live stream

Il design offline del testing replica 1:1 il routing live, prevenendo fenomeni di *data leakage*. Tuttavia, l’ambiente di deployment espone alla minaccia di *data poisoning*. Un aggressore persistente potrebbe infiltrarsi immettendo attacchi lenti per abituare il grafo ai propri comportamenti nocivi, sfalsando lentamente lo scoring e la similarità del coseno. Il problema è mitigato dal gate anti-avvelenamento: sia l’internal state della `TGNMemory` che le code a scorrimento dei vicini temporali vengono aggiornate e *committate* in RAM solo nel caso in cui lo score della rete si posizioni sotto il limite calcolato per le minacce ($< threshold$). I pacchetti classificati come anomalia, inviati a OPA, vengono ignorati dal sistema di Intelligenza Artificiale, preservando incontaminato lo stato vettoriale del modello.

Capitolo 2

Evoluzione del modello: il nodo *Configuration* (schema v4)

2.1 Motivazione

Il modello descritto nel capitolo precedente (schema v3) modella ogni richiesta come una catena causale a tre archi su quattro ruoli di nodo, $source \rightarrow device \rightarrow user \rightarrow resource$. In quello schema il *fingerprint* TLS del client (JA3) è soltanto un *bit di validità* (`features[0]`): segnala se l'handshake è regolare, ma non è un'identità. Questo lascia scoperti due vettori d'attacco centrali nella ZTA:

- *Movimento laterale con tooling nuovo*: un dispositivo già noto, compromesso, inizia a parlare con un client/tool mai osservato su quella macchina. La validità TLS resta intatta (il tool usa una libreria regolare), ma il *fingerprint* è nuovo.
- *Furto di credenziali*: un attaccante che riusa le credenziali della vittima da un proprio client presenta un JA3 *diverso* da quello abituale dell'utente, pur con ruolo/clearance leciti (l'attaccante eredita i permessi della vittima).

Entrambi sono *signal-clean* e *policy-clean*: indistinguibili sintatticamente dal traffico benigno se non per l'identità della configurazione. Da qui la promozione della configurazione del client a **quinto ruolo di nodo**.

2.2 Architettura del nodo *Configuration*

La configurazione è identificata da `conf:<ja3>` quando il *fingerprint* è disponibile, altrimenti dal generico `conf:guest` (sempre presente lato serving, default server-side). Il JA3 mantiene quindi un *doppio ruolo*, ortogonale: resta il bit di validità TLS in `features[0]` (catturato dalla *Feature head*) e diventa un'identità di nodo (catturata dalla memoria temporale e dalle *history features*). Per costruzione la dimensione del messaggio (`msg_dim=7`) e quella delle feature statiche di nodo (`node_feat_dim=16`) restano invariate: il nodo config, come i *source*, non porta attributi statici dedicati ma solo memoria ricorrente, hashed-identity e slot di trust. Lo schema dell'evento passa a `schema_version=4` (i checkpoint v1/v2/v3 sono rifiutati al caricamento).

2.2.1 Topologia: la catena causale a 5 archi

Il nodo config è *inserito tra source e device* nella catena causale, sostituendo l'arco diretto $source \rightarrow device$ con la coppia $source \rightarrow config \rightarrow device$, e aggiungendo un *binding* discriminante $config \rightarrow user$. Il cammino più lungo è quindi:

$$source \rightarrow config \rightarrow device \rightarrow user \rightarrow resource.$$

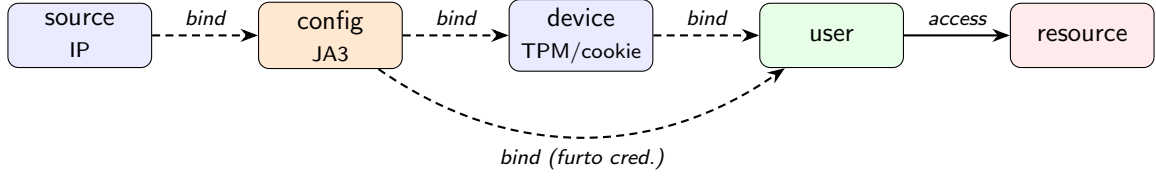


Figura 2.1: Catena causale dello schema v4. I *binding edge* (tratteggiati) portano messaggio-zero; il solo *access edge* $user \rightarrow resource$ porta il messaggio a 7 dimensioni. Lo score dell’evento è il *massimo* sui suoi archi. L’arco diretto $source \rightarrow device$ di v3 è sostituito da $source \rightarrow config \rightarrow device$; il *binding* $config \rightarrow user$ discrimina il furto di credenziali (un client diverso da quello abituale della vittima).

Arco	Tipo	Stato
$user \rightarrow resource$	access (msg 7-dim)	esistente
$device \rightarrow user$	binding (msg-zero)	esistente
$source \rightarrow config$	binding (msg-zero)	nuovo (v4)
$config \rightarrow device$	binding (msg-zero)	nuovo (v4)
$config \rightarrow user$	binding (msg-zero)	nuovo (v4)

Tabella 2.1: Edge set per richiesta nello schema v4 (config sempre presente lato serving). Ordine di commit in memoria, identico in training/replay/serving: $source \rightarrow config$, $config \rightarrow device$, $config \rightarrow user$, $device \rightarrow user$, $user \rightarrow resource$. Fallback coerenti con la gestione opzionale già esistente: device assente $\Rightarrow$ $config \rightarrow user$ fa da ponte; source assente $\Rightarrow$ si salta solo $source \rightarrow config$; per dataset senza config (es. LANL) il *gating has_config* ripristina il percorso legacy $source \rightarrow device$.

L’architettura core (memoria ricorrente, *GraphAttentionEmbedding* multi-hop, *Dual Head Scorer*) è *invariata*: il modello è agnostico al numero di archi per evento, e i nuovi *binding* riusano la stessa primitiva di scoring degli archi esistenti (con *history features* senza ausiliario, come per $device \rightarrow user$). Il generatore sintetico è esteso coerentemente: ogni macchina riceve 1–2 configurazioni abituali da un pool condiviso; il movimento laterale presenta con probabilità 0.5 un *tool* (config) mai usato da quel device; il furto di credenziali alloca al client attaccante un JA3 mai visto (`conf:atk-NNN`).

2.3 Risultati: il contributo del nodo config

La valutazione segue lo stesso protocollo del capitolo precedente (replay per-evento attraverso l’esatto codice di serving, split cronologico 70/10/20, soglia cost-sensitive instradata). Si riportano due confronti complementari: (i) il *deployable* v3 vs v4 all’*operating point* pubblicato (singola run, 200 000 eventi / 15 epoche, `seed=42`), e (ii) la *validazione mirata* del nodo config — un’ablazione controllata *a parità di dati* su seed multipli.

2.3.1 Confronto del modello pubblicato (v3 $\rightarrow$ v4)

La Tabella 2.2 confronta l’artefatto deployabile prima e dopo l’introduzione del nodo config, al medesimo *operating point* (decisione cost-sensitive instradata sul segnale). Il guadagno più rilevante è sul **movimento laterale**: il *recall* operativo passa da $\sim 25.5\%$ a $\sim 59.9\%$ (più che raddoppiato), con AP $0.464 \rightarrow 0.614$ e AUC $0.894 \rightarrow 0.936$, a fronte di un FPR benigno quasi invariato ($\sim 4.8\% \rightarrow \sim 5.6\%$). È il comportamento atteso per costruzione: $config \rightarrow device$ cattura il “tool nuovo su device noto” e $config \rightarrow user$ il “client che quell’utente non usa abitualmente”, cioè i *tell* del laterale e del furto di credenziali.

Metrica (test set)	v3 (4 nodi)	v4 (nodo config)	Δ
AUC aggregata	0.952	0.964	+0.012
AP aggregata	0.896	0.905	+0.009
Recall aggregata (intradata)	0.709	0.835	+0.126
Lateral — AUC	0.894	0.936	+0.042
Lateral — AP	0.464	0.614	+0.150
Lateral — Recall (intradata)	0.255	0.599	+0.344
Lateral — Recall (@1%FPR)	0.141	0.152	+0.011
Policy — AUC / Recall@thr	~ 0.95 / –	0.967 / 0.941	$\sim$
Contextual — AUC / Recall@thr	~ 0.99 / –	0.990 / 0.970	$\sim$
FPR benigno (intradato)	~ 0.048	~ 0.056	+0.008

Tabella 2.2: Confronto del modello *deployabile* v3 vs v4 sul medesimo test set sintetico (replay per-evento, `num_events`=200 000, `seed`=42, decisione cost-sensitive intradata). *Nota metodologica*: valori da *singola* run (training GPU non-deterministico, ± 0.01 – 0.03 per run sul lateral AUC); il segnale robusto è il salto di *recall* operativo del laterale (+0.344), ben oltre il rumore single-run. Le classi *policy* (OPA-owned) e *contextual* (rule-trivial) restano invariate per costruzione. Il confronto con le baseline non-relazionali (Tab. 1.1) è immutato: nessuna baseline modella il nodo config, quindi il vantaggio di v4 si *somma* a quello già documentato del TGN sul segnale temporale.

2.3.2 Validazione mirata: il furto di credenziali

L’*operating point* pubblicato non permetteva di misurare il furto di credenziali: gli slot attaccante del generatore si esauriscono presto (configurazione di default `num_theft_slots`=64, `p_cred_theft`=0.0012 $\Rightarrow$ tutti gli incidenti partono entro $\sim$ l’evento 53k, dentro il 70% di *train*), lasciando *zero* incidenti di furto e *zero* cookie-wipe nello split di test. Per quantificare il contributo specifico del binding *config* $\rightarrow$ *user* si è quindi eseguita una **validazione mirata** su uno stream *theft-rich* (slot e tasso d’incidente aumentati *solo per questa valutazione*, così che gli incidenti si distribuiscano sull’intero stream incluso il test), confrontando — *a parità di dati*, su 3 `seed` (media $\pm$ dev.std) e con `save=False` (l’artefatto in `public/` non viene toccato) — il modello v4 completo contro la sua *ablazione del nodo config* (il modello ricade sul percorso legacy *source* $\rightarrow$ *device*, cioè $\approx$ v3 sugli stessi dati). I risultati sono in Tabella 2.3.

2.3.3 Sweep architetturale

L’introduzione di un quinto nodo (e di due archi aggiuntivi per richiesta) suggerisce di verificare se al modello serva *più capacità*. Si sono valutate tre direzioni — un layer nascosto aggiuntivo nell’MLP della *Feature head* (`link_pred_hidden_layers`=3), una memoria ricorrente più ampia (`memory_dim`=384) e più teste di attenzione nei *TransformerConv* (`gnn_heads`=8) — più la loro combinazione, sempre su 3 `seed` con `save=False`. La Tabella 2.4 riporta l’esito.

2.3.4 Sintesi

Il nodo *configuration* (v4) migliora sostanzialmente il movimento laterale (recall operativo +0.344 all’*operating point* pubblicato) e abilita la rilevazione del furto di credenziali (recall +0.108 vs l’ablazione a parità di dati), senza costi architetturali: `msg_dim` e `node_feat_dim` restano invariati, e lo sweep conferma che la capacità attuale è già adeguata. Il vantaggio si *somma* a

Metrica (test, theft-rich)	v4 (nodo config)	$\approx$ v3 (ablazione)	Δ
Furto credenziali — AUC	0.729 ± 0.088	0.676 ± 0.036	+0.053
Furto credenziali — Recall@thr	0.314 ± 0.107	0.206 ± 0.039	+0.108
Lateral — AUC	0.919 ± 0.018	0.913 ± 0.007	+0.006
Lateral — Recall@thr	0.469 ± 0.088	0.334 ± 0.014	+0.135
AUC aggregata	0.958 ± 0.011	0.956 ± 0.005	+0.002
FPR benigno (cookie-wipe)	0.060 ± 0.021	0.038 ± 0.033	+0.022

Tabella 2.3: Validazione mirata del nodo config (3 seed [42,7,123], 80 000 eventi / 12 epoche, stream *theft-rich*: slot 400/120, `p_theft=0.002`, `p_wipe=0.0008`; `save=False`). Lo split mirato porta finalmente $n = 452$ eventi di furto nel test (erano 0). L’ablazione disattiva il nodo config a parità di stream (percorso legacy *source*→*device*). Il nodo config aumenta il *recall* sul furto credenziali (+0.108) e sul laterale (+0.135); l’AUC sul furto migliora (+0.053) ma con dev.std ampia (conteggi per-seed modesti), quindi il segnale robusto è il *recall* e il fatto che la detection diventi *possibile*. L’aggregata è invariata. Si osserva un lieve aumento del FPR benigno sui cookie-wipe (+0.022, entro le bande di dev.std): il binding *config*→*user* è leggermente più sensibile sui dispositivi re-keyed, trade-off accettabile dato il guadagno sul recall.

Variante	Lateral AUC	Lateral AP	Lateral Rec@thr	agg AUC
baseline v4	0.929 ± 0.009	0.645 ± 0.023	0.607 ± 0.060	0.966 ± 0.005
+1 layer MLP	0.931 ± 0.004	0.641 ± 0.035	0.560 ± 0.037	0.965 ± 0.005
+memory (384)	0.905 ± 0.015	0.543 ± 0.051	0.441 ± 0.050	0.957 ± 0.006
+heads (8)	0.896 ± 0.037	0.558 ± 0.130	0.459 ± 0.161	0.956 ± 0.015
+mem+heads+layer	0.913 ± 0.025	0.526 ± 0.105	0.443 ± 0.111	0.958 ± 0.011

Tabella 2.4: Sweep architetturale (3 seed [42,7,123], 40 000 eventi / 12 epoche, `save=False`). Nessun aumento di capacità migliora la classe *model-owned* (movimento laterale): il layer MLP extra è *neutro* sull’AUC (+0.002, entro la dev.std) ma riduce il *recall* operativo; memoria più ampia e più teste *peggiorano* AUC/AP/recall e *aumentano* la varianza tra seed (es. `gnn_heads=8`: recall 0.459 ± 0.161). Il collo di bottiglia del laterale non è la capacità parametrica ma il *segnale* (history features + memoria temporale), coerentemente con l’ablation a 3 seed del Cap. 1 (history features +0.163 AUC, dominante). **Decisione:** si mantiene l’architettura corrente; il *deployable* non viene modificato.

quello già documentato del TGN sulle baseline non-relazionali (Tab. 1.1), che non modellano la configurazione del client.

2.4 Esperimento: granularità dell’identità del dispositivo (nodo `dev:guest`)

2.4.1 Ipotesi e motivazione

Nello schema v4 il nodo *device* è identificato da `tpm:<id>` quando il TPM è attestato e, in assenza di TPM, da un *cookie persistente per-macchina* (`ck:<id>`), con re-keying su un nodo “freddo” a ogni *cookie-wipe*. Ogni macchina senza TPM è quindi un *nodo distinto* del grafo. In analogia con il nodo *configuration* — dove un *fingerprint* non risolvibile ricade sempre sul

generico `conf:guest` (Sez. 2) — ci si è chiesti se convenga *collassare tutti i dispositivi senza TPM su un unico nodo condiviso* `dev:guest`, anonimo e a bassa fiducia, invece di mantenere un’identità-cookie per macchina. L’ipotesi indaga un *trade-off di modellazione*: l’identità per-cookie aggiunge potere discriminante (un device mai visto è una *novelty cue*) oppure è soltanto *rumore*, dato che gli attacchi *rule-blind* (laterale, furto credenziali) sono per costruzione *signal-clean* e il loro segnale è già portato dalle edge *config/source/user*?

Il comportamento è esposto da un flag di configurazione (`guest_device_fallback`, default *off* = keying per-cookie attuale): quando attivo, ogni device non-TPM (cookie, IP-debole o assente) collassa su `dev:guest` sia nel generatore (instradamento sul medesimo nodo, poiché in addestramento i nodi sono indici di *slot*, non chiavi) sia lato serving (collasso della chiave esterna), così che addestramento e *serving* restino coerenti. Coerentemente, il *cookie-wipe* viene *neutralizzato*: un nodo guest condiviso non possiede un cookie per-macchina da resettare. È importante notare che, sotto questa politica, *anche il dispositivo dell’attaccante* nel furto di credenziali (privo di TPM) collassa su `dev:guest`: il *tell* di “device mai visto” sparisce e la rilevazione deve appoggiarsi unicamente all’IP e al JA3 mai visti e al *binding config→user*.

2.4.2 Protocollo e risultati

Si confronta — a parità di impostazioni del generatore, su 3 *seed* [42,7,123] (*media±dev.std*), con `save=False` e sullo stesso stream *theft-rich* della validazione del nodo config (80 000 eventi / 12 epoche, slot 400/120, `p_theft=0.002`, `p_wipe=0.0008`) — il *keying* per-cookie (*baseline*) contro il collasso su `dev:guest`. La Tabella 2.5 riporta l’esito.

Metrica (test, theft-rich)	cookie (baseline)	dev:guest	Δ
Lateral — AUC	0.915 ± 0.019	0.911 ± 0.007	−0.003
Lateral — AP	0.583 ± 0.068	0.632 ± 0.017	+0.049
Lateral — Recall@thr	0.453 ± 0.060	0.488 ± 0.019	+0.035
Furto credenziali — AUC	0.701 ± 0.078	0.804 ± 0.024	+0.103
Furto credenziali — Recall@thr	0.244 ± 0.082	0.287 ± 0.062	+0.043
AUC aggregata	0.954 ± 0.013	0.961 ± 0.001	+0.007
Recall aggregata (instradata)	0.764 ± 0.036	0.787 ± 0.006	+0.023
FPR benigno (instradato)	0.045 ± 0.002	0.037 ± 0.004	−0.007
FPR cookie-wipe	0.041 ± 0.013 (<i>n</i> =651)	— (<i>n</i> =0)	neutralizzato

Tabella 2.5: Esperimento sull’identità del device: *keying* per-cookie vs collasso su `dev:guest` (3 *seed* [42,7,123], 80 000 eventi / 12 epoche, stream *theft-rich*, `save=False`). Le due varianti condividono *seed* e impostazioni ma *non* sono appaiate evento-per-evento: la neutralizzazione del *cookie-wipe* altera il percorso del generatore di numeri casuali (da cui il diverso conteggio di positivi: *lateral* *n*=3959 vs 4032, furto *n*=452 vs 386 sui 3 *seed*); il confronto è quindi *distribuzionale*. Il collasso su `dev:guest` non degrada la detection (lateral AUC piatto, −0.003 entro la *dev.std*) e anzi migliora il furto credenziali, abbassa l’FPR benigno, *riduce marcatamente la varianza tra seed* ed elimina l’intera classe di falsi positivi da cookie-wipe (*n_{wiped}*=0, conferma della neutralizzazione implementata).

2.4.3 Interpretazione

Il risultato è *controintuitivo*: rimuovere l’identità per-macchina dei dispositivi senza TPM non solo non danneggia la rilevazione, ma costituisce un *miglioramento di Pareto* sulle metriche

misurate. La lettura è coerente con l’architettura del modello:

- **L’identità per-cookie era rumore netto.** I nodi-cookie per-macchina sono *sparsi*: ciascuno vede pochi eventi, quindi accumula una memoria ricorrente e dei contatori di storia deboli e freddi. Concentrare tutto il traffico non-TPM su un solo nodo `dev:guest` produce *binding config*→*device* e *device*→*user* statisticamente più stabili, riducendo i falsi positivi benigni (-0.007 FPR) e — in modo netto — la varianza tra *seed* (es. laterale AUC ± 0.007 vs ± 0.019 ; furto AUC ± 0.024 vs ± 0.078).
- **Il segnale degli attacchi non risiedeva nel device.** Laterale e furto credenziali sono *signal-clean* per costruzione: la *novelty cue* vive sulle *history features* delle edge *user/config/source*, non sulla novità del nodo-device. Per il furto, pur perdendo il *tell* di “device mai visto” (l’attaccante collassa anch’esso su `dev:guest`), restano l’IP e il JA3 mai visti e il *binding config*→*user* (già documentato come discriminante principale del furto, Sez. 2.3); il netto guadagno di AUC del furto ($+0.103$) riflette soprattutto la *riduzione di varianza* rispetto a una *baseline* ad alta dev.std (± 0.078).
- **Eliminazione di una classe di falsi positivi.** Senza identità-cookie non esiste il *re-keying* su nodo freddo: l’intera classe di falsi positivi da *cookie-wipe* (FPR ~ 0.041 su $n=651$ nella *baseline*) scompare ($n_{\text{wiped}}=0$).

Limiti e costo non misurato. Il guadagno sulle metriche di detection va soppesato con un costo che esse *non* catturano: collassando i dispositivi senza TPM si perde l’*attribuzione per-macchina* (tracciabilità forense, capacità di isolare un singolo endpoint compromesso, audit). Inoltre il confronto è distribuzionale e su stream sintetico: il Δ sull’AUC laterale è entro il rumore e la quota di guadagno dovuta alla riduzione di varianza non è separabile da un eventuale guadagno “vero”. Per questi motivi il *deployable* mantiene il *default* a *keying* per-cookie (`guest_device_fallback=False`); il collasso su `dev:guest` resta una politica attivabile e un’evidenza utile sul fatto che, su questo stream, l’identità fine del dispositivo *non* è il portatore del segnale.